

EEG Eye State Dataset

The **EEG Eye State** dataset contains electroencephalogram (EEG) readings from multiple scalp electrodes recorded over time from a single participant. EEG (Electroencephalography) is a technique for recording the brain’s electrical activity from the scalp using electrodes. The task is to predict the subject’s **eye state**, whether their eyes are **open** or **closed**, based on instantaneous EEG measurements using a Multi-Layer Perceptron.

Source:

Originally from the **UCI Machine Learning Repository**, this dataset is widely used for binary classification experiments in brain-computer interface (BCI) research.

- [UCI Dataset Page](#)

Target Variable:

- eyeDetection** (*binary*): 0 → Eyes open 1 → Eyes closed

Features:

Column Name	Description	Data Type
AF3	EEG signal from frontal electrode AF3 (prefrontal area, left hemisphere)	Numeric (float)
F7	EEG signal from frontal electrode F7 (frontal lobe, left side)	Numeric (float)
F3	EEG signal from frontal electrode F3 (motor planning / attention, left hemisphere)	Numeric (float)
FC5	EEG signal from fronto-central electrode FC5	Numeric (float)
T7	EEG signal from temporal electrode T7 (auditory/language area, left side)	Numeric (float)
P7	EEG signal from parietal electrode P7 (spatial processing, left hemisphere)	Numeric (float)

Column Name	Description	Data Type
O1	EEG signal from occipital electrode O1 (visual cortex, left hemisphere)	<i>Numeric (float)</i>
O2	EEG signal from occipital electrode O2 (visual cortex, right hemisphere)	<i>Numeric (float)</i>
P8	EEG signal from parietal electrode P8 (spatial processing, right hemisphere)	<i>Numeric (float)</i>
T8	EEG signal from temporal electrode T8 (auditory/language area, right side)	<i>Numeric (float)</i>
FC6	EEG signal from fronto-central electrode FC6	<i>Numeric (float)</i>
F4	EEG signal from frontal electrode F4 (motor planning / attention, right hemisphere)	<i>Numeric (float)</i>
F8	EEG signal from frontal electrode F8 (frontal lobe, right side)	<i>Numeric (float)</i>
AF4	EEG signal from frontal electrode AF4 (prefrontal area, right hemisphere)	<i>Numeric (float)</i>
eyeDetection	Eye state label: 0 (Open), 1 (Closed)	Categorical (target)

Additional Library to Install: `liac-arff`

This library lets us read and write `.arff` files (**Attribute-Relation File Format**), which are a type of text-based dataset format.

```
In [85]: !pip install -q liac-arff
```

Read the arff file and load into pandas dataframe

```
In [86]: import arff
import pandas as pd

# Load the ARFF file
with open("EEG Eye State.arff") as f:
    dataset = arff.load(f)

#Convert to DataFrame
df = pd.DataFrame(dataset['data'], columns=[attr[0] for attr in dataset['attributes']])
df
```

Out[86]:

	AF3	F7	F3	FC5	T7	P7	O1	O2	P8	T8	FC6	F4	F8	AF4	e
0	4329.23	4009.23	4289.23	4148.21	4350.26	4586.15	4096.92	4641.03	4222.05	4238.46	4211.28	4280.51	4635.90	4393.85	
1	4324.62	4004.62	4293.85	4148.72	4342.05	4586.67	4097.44	4638.97	4210.77	4226.67	4207.69	4279.49	4632.82	4384.10	
2	4327.69	4006.67	4295.38	4156.41	4336.92	4583.59	4096.92	4630.26	4207.69	4222.05	4206.67	4282.05	4628.72	4389.23	
3	4328.72	4011.79	4296.41	4155.90	4343.59	4582.56	4097.44	4630.77	4217.44	4235.38	4210.77	4287.69	4632.31	4396.41	
4	4326.15	4011.79	4292.31	4151.28	4347.69	4586.67	4095.90	4627.69	4210.77	4244.10	4212.82	4288.21	4632.82	4398.46	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
14975	4281.03	3990.26	4245.64	4116.92	4333.85	4614.36	4074.87	4625.64	4203.08	4221.54	4171.28	4269.23	4593.33	4340.51	
14976	4276.92	3991.79	4245.13	4110.77	4332.82	4615.38	4073.33	4621.54	4194.36	4217.44	4162.56	4259.49	4590.26	4333.33	
14977	4277.44	3990.77	4246.67	4113.85	4333.33	4615.38	4072.82	4623.59	4193.33	4212.82	4160.51	4257.95	4591.79	4339.49	
14978	4284.62	3991.79	4251.28	4122.05	4334.36	4616.41	4080.51	4628.72	4200.00	4220.00	4165.64	4267.18	4596.41	4350.77	
14979	4287.69	3997.44	4260.00	4121.03	4333.33	4616.41	4088.72	4638.46	4212.31	4226.67	4167.69	4274.36	4597.95	4350.77	

14980 rows × 15 columns



Column names with their data typs

In [87]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14980 entries, 0 to 14979
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   AF3              14980 non-null  float64
1   F7               14980 non-null  float64
2   F3               14980 non-null  float64
3   FC5              14980 non-null  float64
4   T7               14980 non-null  float64
5   P7               14980 non-null  float64
6   O1               14980 non-null  float64
7   O2               14980 non-null  float64
8   P8               14980 non-null  float64
9   T8               14980 non-null  float64
10  FC6              14980 non-null  float64
11  F4               14980 non-null  float64
12  F8               14980 non-null  float64
13  AF4              14980 non-null  float64
14  eyeDetection     14980 non-null  object
dtypes: float64(14), object(1)
memory usage: 1.7+ MB
```

Checking for missing values

```
In [88]: df.isna().sum()
```

Out[88]:

	0
AF3	0
F7	0
F3	0
FC5	0
T7	0
P7	0
O1	0
O2	0
P8	0
T8	0
FC6	0
F4	0
F8	0
AF4	0
eyeDetection	0

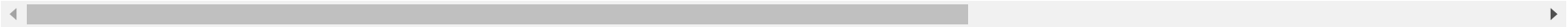
dtype: int64

Descriptive Statistics of Numerical Attributes

```
In [89]: df.describe()
```

Out[89]:

	AF3	F7	F3	FC5	T7	P7	O1	O2	
count	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.
mean	4321.917777	4009.767694	4264.022433	4164.946326	4341.741075	4644.022379	4110.400160	4616.056904	4218.
std	2492.072174	45.941672	44.428052	5216.404632	34.738821	2924.789537	4600.926543	29.292603	2136.
min	1030.770000	2830.770000	1040.000000	2453.330000	2089.740000	2768.210000	2086.150000	4567.180000	1357.
25%	4280.510000	3990.770000	4250.260000	4108.210000	4331.790000	4611.790000	4057.950000	4604.620000	4190.
50%	4294.360000	4005.640000	4262.560000	4120.510000	4338.970000	4617.950000	4070.260000	4613.330000	4199.
75%	4311.790000	4023.080000	4270.770000	4132.310000	4347.180000	4626.670000	4083.590000	4624.100000	4209.
max	309231.000000	7804.620000	6880.510000	642564.000000	6474.360000	362564.000000	567179.000000	7264.100000	265641.

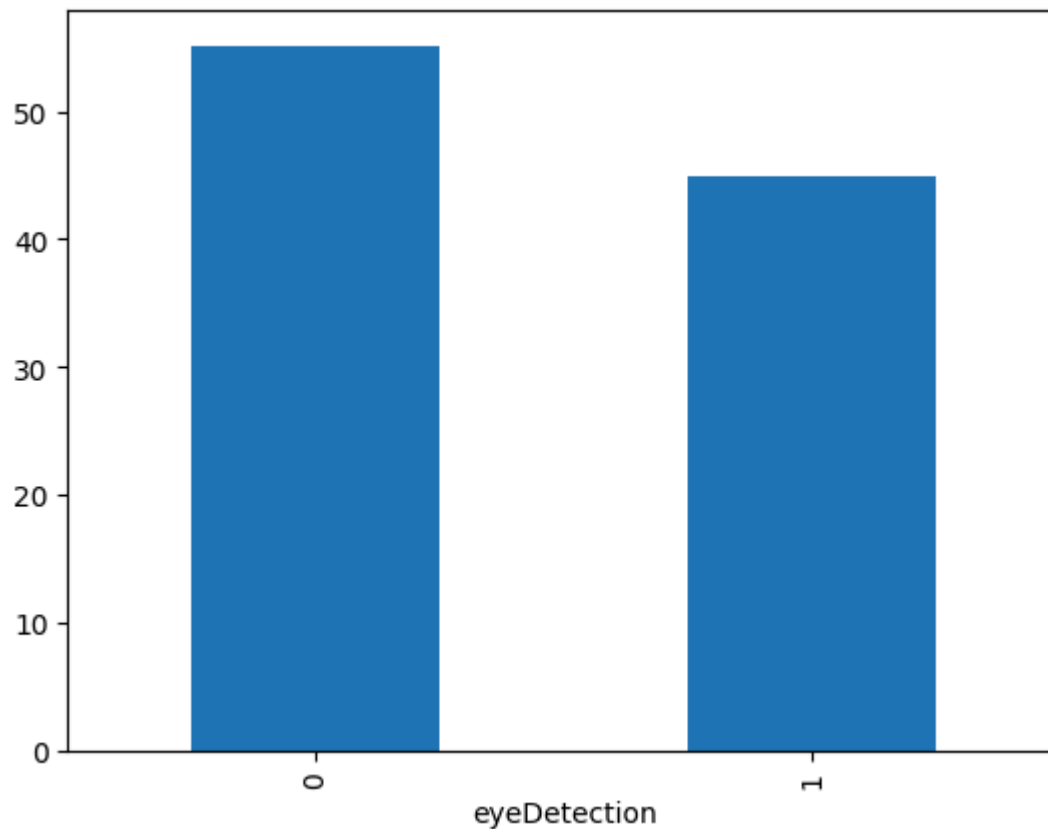


Count plot for target variable

```
In [90]: print(df['eyeDetection'].value_counts(normalize=True))
df['eyeDetection'].value_counts(normalize=True).mul(100).plot(kind='bar')
```

eyeDetection
0 0.551202
1 0.448798
Name: proportion, dtype: float64

Out[90]: <Axes: xlabel='eyeDetection'>



Preparing the data

- train/test split
- standardizing features(zero mean, unit variance)

```
In [91]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop('eyeDetection', axis=1).values
y = df['eyeDetection'].astype(int).values

print(f"Shape: {X.shape} {y.shape}")
```

```
# 80/20 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Train set size")
print(X_train_scaled.shape, y_train.shape)
print("Test set size")
print(X_test_scaled.shape, y_test.shape)
```

Shape: (14980, 14) (14980,)

Train set size

(11984, 14) (11984,)

Test set size

(2996, 14) (2996,)

Multi-Layer Perceptron

Pytorch tensors

In [92]: `torch.manual_seed(42)`

```
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)
```

Tuning for the choice of optimizer

- Adam with lr 0.001 and 0.01

- SGD with lr 0.001 and 0.01

```
In [93]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
)
import math
import matplotlib.pyplot as plt
torch.manual_seed(42)

class MLP(nn.Module):
    def __init__(self, input_dim, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1) # logits
        )
    def forward(self, x):
        return self.net(x)

def make_optimizer(model, name, lr):
    if name.lower() == "adam":
        return optim.Adam(model.parameters(), lr=lr)
    elif name.lower() == "sgd":
        return optim.SGD(model.parameters(), lr=lr)
    else:
        raise ValueError(name)

# train/eval
def train_and_eval_opt(optimizer_name, lr, epochs=200):
    model = MLP(X_train_tensor.shape[1], hidden=64)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = make_optimizer(model, optimizer_name, lr)
```

```

train_losses, test_losses = [], []
for _ in range(epochs):
    model.train()
    optimizer.zero_grad()
    logits = model(X_train_tensor)
    loss = criterion(logits, y_train_tensor)
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

    model.eval()
    with torch.no_grad():
        test_logits = model(X_test_tensor)
        test_loss = criterion(test_logits, y_test_tensor).item()
    test_losses.append(test_loss)

model.eval()
with torch.no_grad():
    logits = model(X_test_tensor)
    probs = torch.sigmoid(logits)
    preds = (probs >= 0.5).float()

y_true = y_test_tensor.cpu().numpy().ravel()
y_pred = preds.cpu().numpy().ravel()
y_proba = probs.cpu().numpy().ravel()

metrics = {
    "Acc": accuracy_score(y_true, y_pred),
    "Prec": precision_score(y_true, y_pred),
    "Rec": recall_score(y_true, y_pred),
    "F1": f1_score(y_true, y_pred),
    "AUC": roc_auc_score(y_true, y_proba),
}
history = {"train_loss": train_losses, "test_loss": test_losses}
return metrics, history

configs = [("SGD", 0.001), ("SGD", 0.01), ("Adam", 0.001), ("Adam", 0.01)]
results, histories = {}, {}

```

```
for opt, lr in configs:
    key = f"{opt}@{lr}"
    metrics, history = train_and_eval_opt(opt, lr, epochs=200)
    results[key] = metrics
    histories[key] = history

df_opt = pd.DataFrame(results).T

def plot_loss_grid(histories, max_cols=2, subtitle="Train/Test Loss vs Optimizer used"):
    keys = list(histories.keys())
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

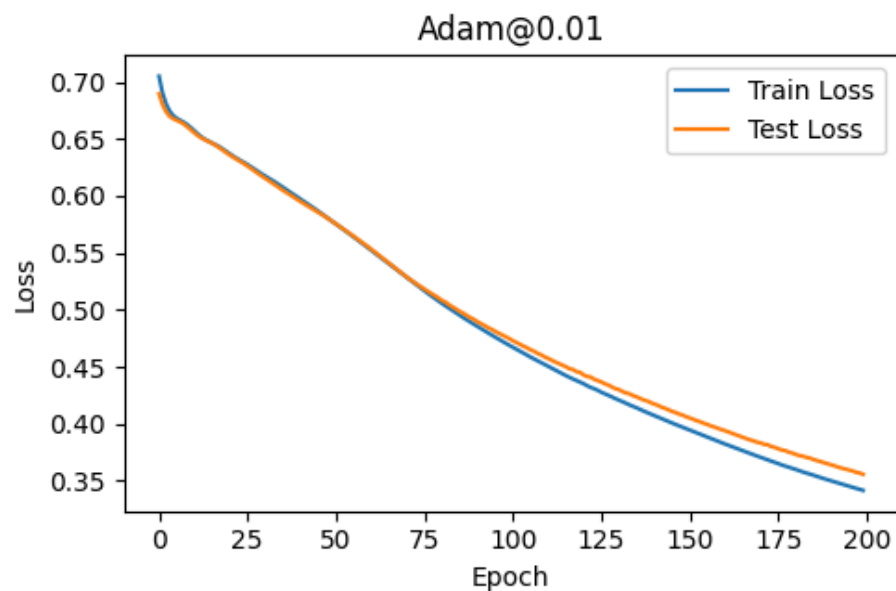
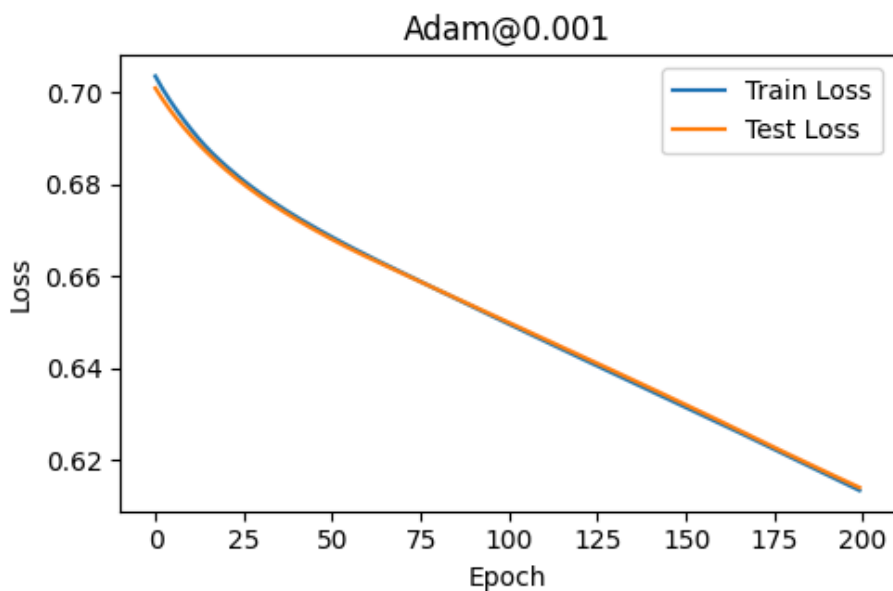
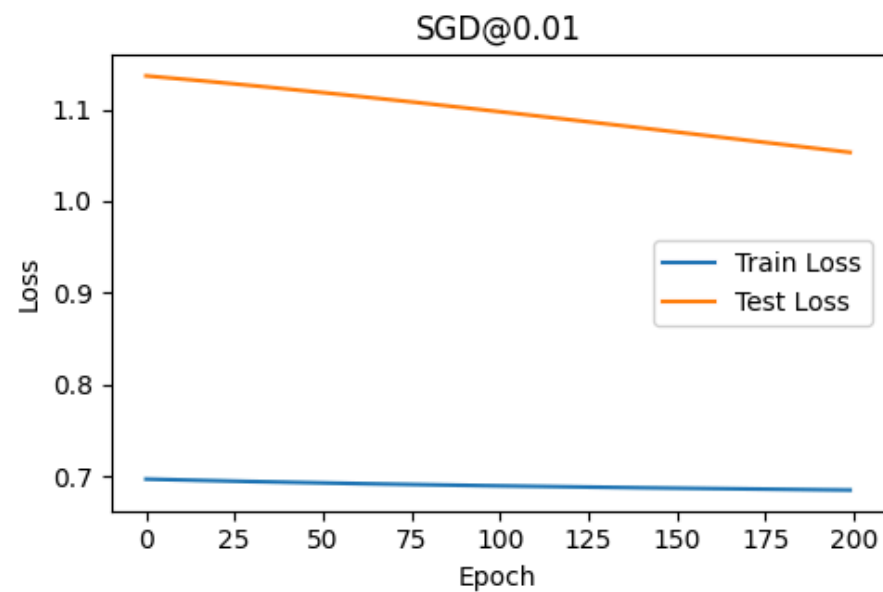
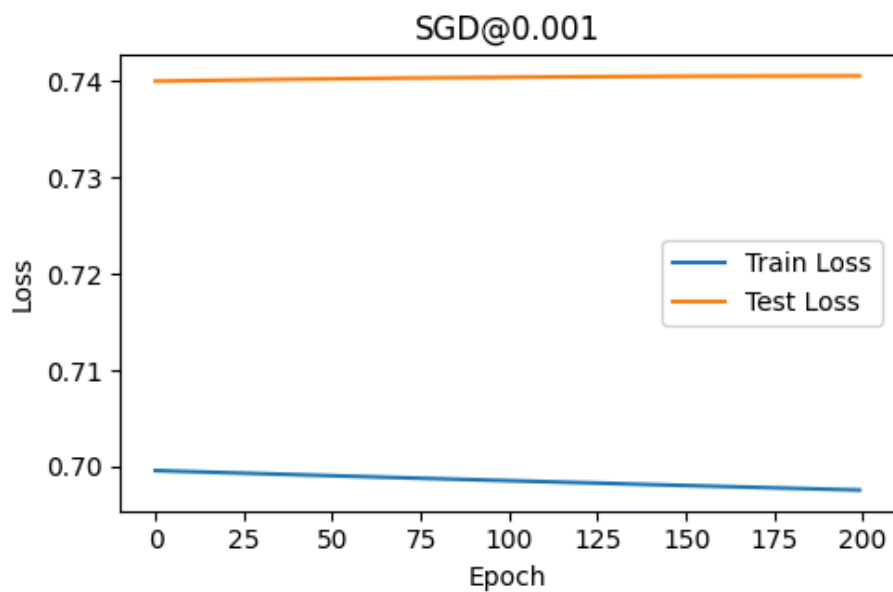
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)
    for idx, k in enumerate(keys):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[k]
        ax.plot(hist["train_loss"], label="Train Loss")
        ax.plot(hist["test_loss"], label="Test Loss")
        ax.set_title(k)
        ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
        ax.legend()

    for k in range(n, rows*cols):
        r, c = divmod(k, cols)
        axes[r][c].axis("off")

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()

plot_loss_grid(histories)
```

Train/Test Loss vs Optimizer used



- **SGD:** Both learning rates show poor convergence, at **lr = 0.001**, loss is almost flat (underfitting), while at **lr = 0.01**, train loss decreases slightly but test loss stays much higher, indicating poor generalization.
- **Adam:** Both learning rates show consistent loss reduction with a small gap between train and test losses, and **lr = 0.01** achieves the fastest convergence and lowest final losses.

```
In [94]: def plot_metric_bars_grid(df_results, rows=3, cols=2, subtitle="Metrics vs Setting"):
    """
    df_results: DataFrame indexed by setting (e.g., 'Adam@0.003')
    Creates a grid of bar plots, one per metric, sized rows×cols.
    """
    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

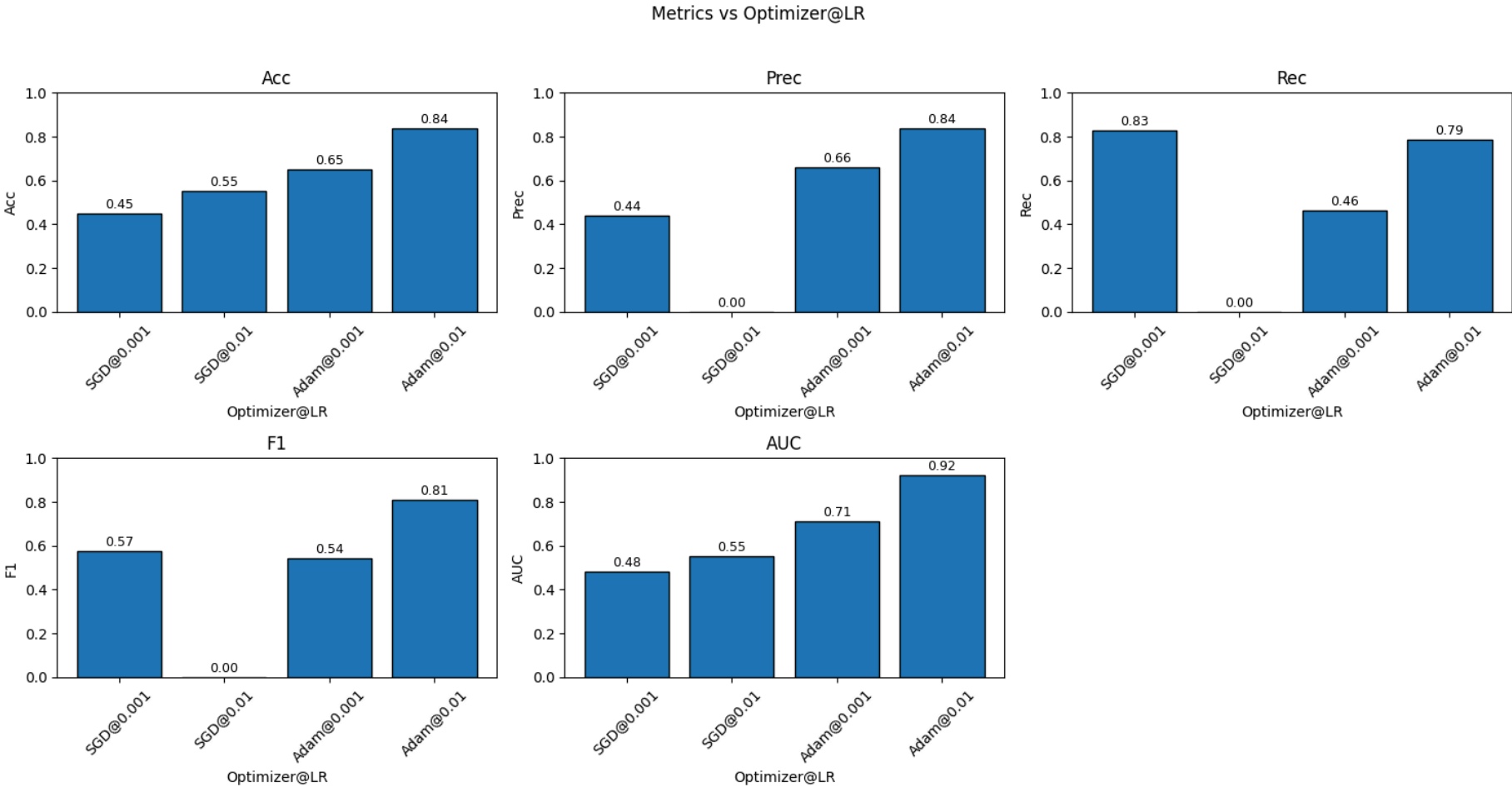
    for i, metric in enumerate(metrics):
        ax = ax_list[i]
        vals = df_results[metric].values
        ax.bar(idx_labels, vals, edgecolor='black')
        ax.set_title(metric)
        ax.set_xlabel("Optimizer@LR")
        ax.set_ylabel(metric)
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)
        for x, v in enumerate(vals):
            ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

    for j in range(len(metrics), rows*cols):
        ax_list[j].axis("off")

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()
```

```
plot_metric_bars_grid(df_opt, rows=2, cols=3, suptitle="Metrics vs Optimizer@LR")

df_opt
```



Out[94]:

	Acc	Prec	Rec	F1	AUC
SGD@0.001	0.449266	0.439699	0.826766	0.574084	0.479884
SGD@0.01	0.550401	0.000000	0.000000	0.000000	0.553743
Adam@0.001	0.650534	0.657505	0.462454	0.542994	0.710034
Adam@0.01	0.835781	0.838761	0.785130	0.811060	0.923535

- The results suggest that Adam works fine with higher learning rate (0.01) convergences without destabilizing training whereas SGD at the same higher learning rate fails to converge.

Tuning for the choice of learning rate

- Adam with lr : [1e-4, 3e-4, 1e-3, 3e-3, 1e-2]

```
In [95]: import os
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
)
import math
import matplotlib.pyplot as plt

torch.manual_seed(42)

class MLP(nn.Module):
    def __init__(self, input_dim, hidden=64):
        super().__init__()
```

```
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1)  # logits
        )
    def forward(self, x):
        return self.net(x)

def train_and_eval(lr, epochs=50):

    model = MLP(X_train_tensor.shape[1], hidden=64)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_losses = []
    test_losses = []

    for _ in range(epochs):
        # train loss
        model.train()
        optimizer.zero_grad()
        logits = model(X_train_tensor)
        loss = criterion(logits, y_train_tensor)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        # test loss
        model.eval()
        with torch.no_grad():
            test_logits = model(X_test_tensor)
            test_loss = criterion(test_logits, y_test_tensor).item()
            test_losses.append(test_loss)

    # evaluation on test set
    model.eval()
    with torch.no_grad():
        logits = model(X_test_tensor)
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.5).float()
```



```

y_true = y_test_tensor.cpu().numpy().ravel()
y_pred = preds.cpu().numpy().ravel()
y_proba = probs.cpu().numpy().ravel()

metrics = {
    "Acc": accuracy_score(y_true, y_pred),
    "Prec": precision_score(y_true, y_pred),
    "Rec": recall_score(y_true, y_pred),
    "F1": f1_score(y_true, y_pred),
    "AUC": roc_auc_score(y_true, y_proba),
}

history = {"train_loss": train_losses, "test_loss": test_losses}
return metrics, history

learning_rates = [1e-4, 3e-4, 1e-3, 3e-3, 1e-2]
results = {}
histories = {}

for lr in learning_rates:
    metrics, history = train_and_eval(lr, epochs=300, )
    results[lr] = metrics
    histories[lr] = history

def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss per Learning Rate"):

    lrs = list(histories.keys())
    n = len(lrs)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)

    for idx, lr in enumerate(lrs):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[lr]
        ax.plot(hist["train_loss"], label="Train Loss")

```

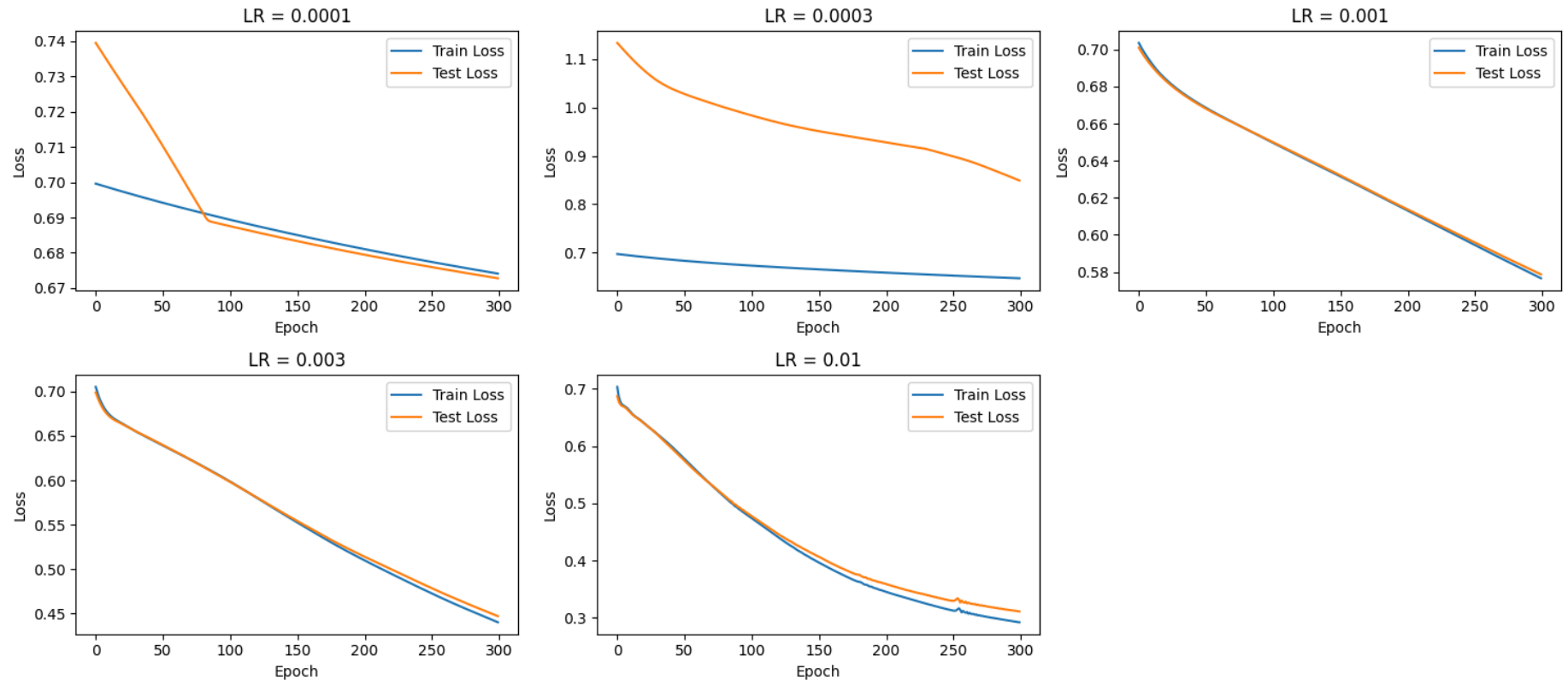
```
ax.plot(hist["test_loss"], label="Test Loss")
ax.set_title(f"LR = {lr}")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.legend()

for k in range(n, rows*cols):
    r, c = divmod(k, cols)
    axes[r][c].axis("off")

fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

plot_loss_grid(histories)
```

Train/Test Loss per Learning Rate



1. Very low learning rates (**0.0001** and **0.0003**) show slow convergence and higher final losses, with **0.0003** also having a large persistent gap between train and test loss, indicating poor generalization.
2. Increasing the learning rate improves both convergence speed and final loss, with **0.01** achieving the fastest convergence and lowest losses while maintaining a small train-test gap.

```
In [96]: df_results = pd.DataFrame.from_records(
    [{ "LR": lr, **results[lr] } for lr in learning_rates]
).set_index("LR")
```

```
import math
import matplotlib.pyplot as plt

def plot_metric_bars_grid(df_results, rows=2, cols=3, subtitle="Metrics vs Learning Rate"):

    metrics = list(df_results.columns)
    lrs = df_results.index.astype(str)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

    for i, metric in enumerate(metrics):
        ax = ax_list[i]
        vals = df_results[metric].values
        ax.bar(lrs, vals)
        ax.set_title(metric)
        ax.set_xlabel("Learning Rate")
        ax.set_ylabel(metric)
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)

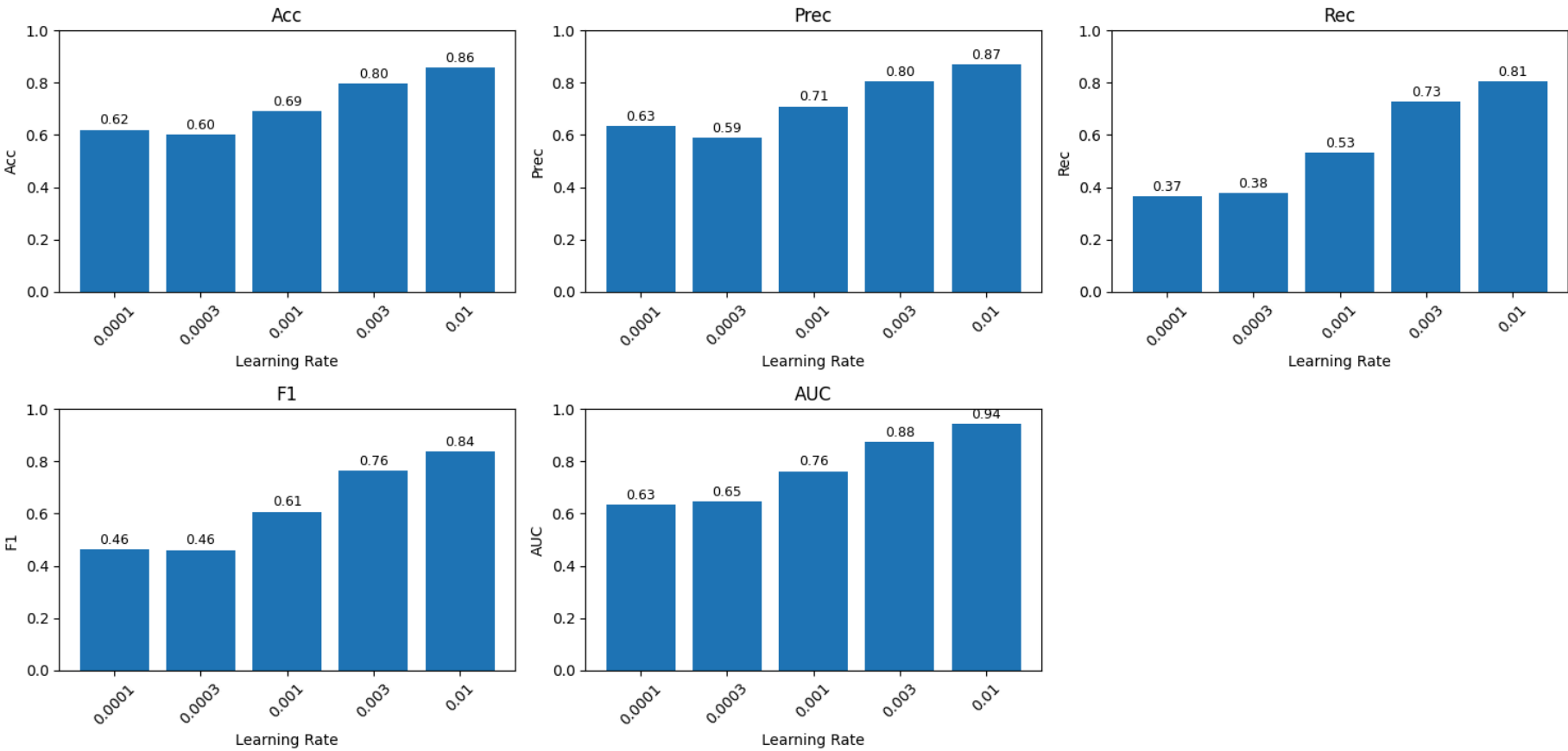
        for x, v in enumerate(vals):
            ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

    for j in range(len(metrics), rows*cols):
        ax_list[j].axis('off')

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()

plot_metric_bars_grid(df_results, rows=2, cols=3)
df_results
```

Metrics vs Learning Rate



Out[96]:

	Acc	Prec	Rec	F1	AUC
LR					
0.0001	0.620160	0.633205	0.365799	0.463713	0.633253
0.0003	0.602804	0.589803	0.378439	0.461051	0.646753
0.0010	0.691923	0.709742	0.530855	0.607401	0.762420
0.0030	0.798398	0.804437	0.727881	0.764247	0.875934
0.0100	0.859146	0.870088	0.806691	0.837191	0.942499

- Higher learning rates with Adam improve both convergence and performance, with 0.01 giving the best overall metrics, while very low learning rates (0.0001, 0.0003) lead to slower learning and significantly lower scores

Tuning for the size of first hidden layer

- sizes: 32, 64, 128, 256

```
In [97]: import os
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
)

torch.manual_seed(42)
```

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1)  # logits
        )
    def forward(self, x):
        return self.net(x)

best_lr = 0.01

def train_and_eval(hidden, epochs=200, lr=best_lr):

    model = MLP(X_train_tensor.shape[1], hidden=hidden)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_losses, test_losses = [], []

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        logits = model(X_train_tensor)
        loss = criterion(logits, y_train_tensor)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            test_logits = model(X_test_tensor)
            test_loss = criterion(test_logits, y_test_tensor).item()
            test_losses.append(test_loss)

    #
    model.eval()
    with torch.no_grad():
        logits = model(X_test_tensor)
        probs = torch.sigmoid(logits)
```

```

        preds = (probs >= 0.5).float()

    y_true = y_test_tensor.cpu().numpy().ravel()
    y_pred = preds.cpu().numpy().ravel()
    y_proba = probs.cpu().numpy().ravel()

    metrics = {
        "Acc": accuracy_score(y_true, y_pred),
        "Prec": precision_score(y_true, y_pred),
        "Rec": recall_score(y_true, y_pred),
        "F1": f1_score(y_true, y_pred),
        "AUC": roc_auc_score(y_true, y_proba),
    }
    history = {"train_loss": train_losses, "test_loss": test_losses}
    return metrics, history

#
hidden_sizes = [32, 64, 128, 256]
results = {}
histories = {}

for h in hidden_sizes:
    metrics, history = train_and_eval(hidden=h, epochs=300)
    results[h] = metrics
    histories[h] = history

df_hidden1 = pd.DataFrame.from_records(
    [{"Hidden1": h, **results[h]} for h in hidden_sizes]
).set_index("Hidden1")

import math
def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss vs First Hidden layer size"):
    keys = list(histories.keys()) # hidden sizes
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)

```



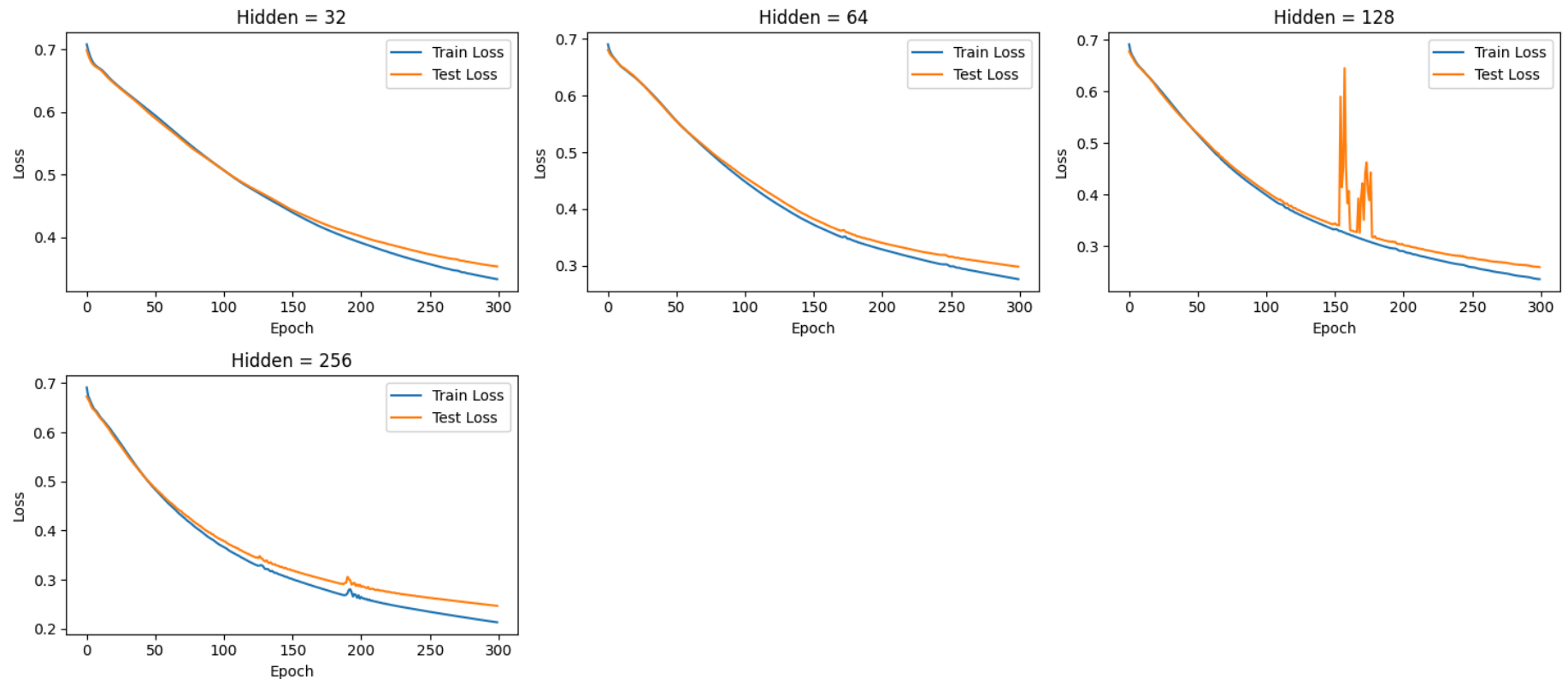
```
for idx, k in enumerate(keys):
    r, c = divmod(idx, cols)
    ax = axes[r][c]
    hist = histories[k]
    ax.plot(hist["train_loss"], label="Train Loss")
    ax.plot(hist["test_loss"], label="Test Loss")
    ax.set_title(f"Hidden = {k}")
    ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
    ax.legend()

for k in range(n, rows*cols):
    r, c = divmod(k, cols)
    axes[r][c].axis("off")

fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

plot_loss_grid(histories)
```

Train/Test Loss vs First Hidden layer size



- Increasing the first hidden layer size generally speeds up convergence and reduces final losses, but very large sizes (128, 256) have small fluctuations in the test loss for 128 and mild spikes for 256.
- Smaller sizes (32, 64) produce smooth and stable curves but converge more slowly and to slightly higher final losses compared to larger hidden sizes.

```
In [98]: def plot_metric_bars_grid(df_results, rows=3, cols=2, subtitle="Metrics vs Setting"):
    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)
```

```

fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
ax_list = axes.ravel()

for i, metric in enumerate(metrics):
    ax = ax_list[i]
    vals = df_results[metric].values
    ax.bar(idx_labels, vals, edgecolor='black')
    ax.set_title(metric)
    ax.set_xlabel("#of neurons")
    ax.set_ylabel(metric)
    ax.set_ylim(0, 1)
    ax.tick_params(axis='x', rotation=45)
    for x, v in enumerate(vals):
        ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

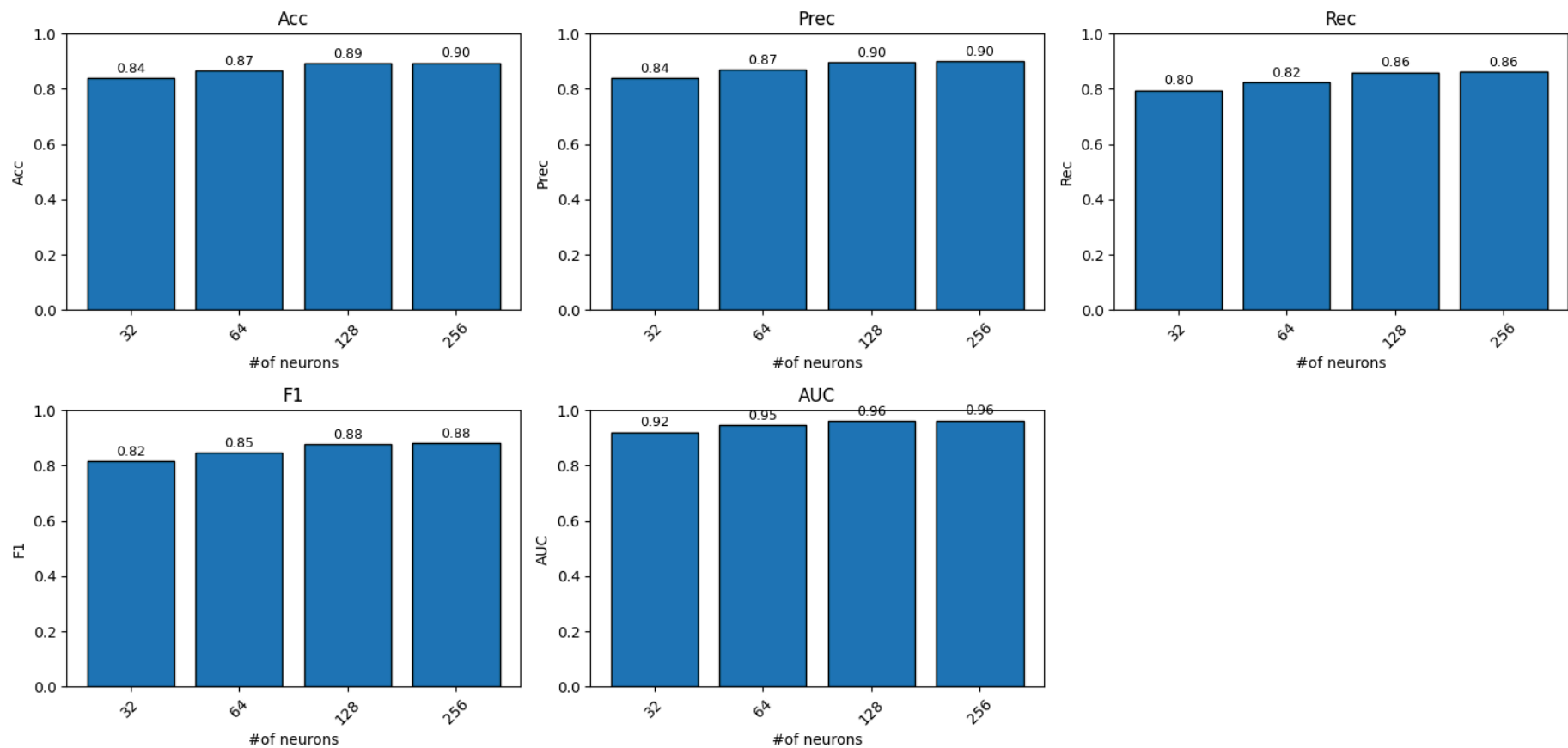
for j in range(len(metrics), rows*cols):
    ax_list[j].axis("off")

fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

plot_metricBars_grid(df_hidden1, rows=2, cols=3, suptitle="Metrics vs First Hidden Layer Size")
df_hidden1

```

Metrics vs First Hidden Layer Size



Out[98]:

	Acc	Prec	Rec	F1	AUC
Hidden1					
32	0.839453	0.838558	0.795539	0.816482	0.921440
64	0.865821	0.870385	0.823792	0.846448	0.947419
128	0.892857	0.896285	0.860967	0.878271	0.961768
256	0.895527	0.901244	0.861710	0.881034	0.964511

Larger hidden layer sizes consistently improve performance, with **256 neurons** giving the highest metrics and second highest with 128 neurons.

Tuning for the size of second hidden layer

- sizes: 16, 32, 64, 128
- lr:0.01

```
In [99]: import os
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
)
import math

torch.manual_seed(42)

class MLP2HL(nn.Module):
    def __init__(self, input_dim, hidden2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, 1) # logits
        )
    def forward(self, x):
```

```
        return self.net(x)

def train_and_eval(hidden, epochs=200, lr=best_lr):

    model = MLP2HL(X_train_tensor.shape[1], hidden2=hidden)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr,)

    train_losses, test_losses = [], []

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        logits = model(X_train_tensor)
        loss = criterion(logits, y_train_tensor)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            test_logits = model(X_test_tensor)
            test_loss = criterion(test_logits, y_test_tensor).item()
            test_losses.append(test_loss)

    model.eval()
    with torch.no_grad():
        logits = model(X_test_tensor)
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.5).float()

    y_true = y_test_tensor.cpu().numpy().ravel()
    y_pred = preds.cpu().numpy().ravel()
    y_proba = probs.cpu().numpy().ravel()

    metrics = {
        "Acc": accuracy_score(y_true, y_pred),
        "Prec": precision_score(y_true, y_pred),
        "Rec": recall_score(y_true, y_pred),
        "F1": f1_score(y_true, y_pred),
        "AUC": roc_auc_score(y_true, y_proba),
```

```

    }
    history = {"train_loss": train_losses, "test_loss": test_losses}
    return metrics, history

# second-layer-size
second_layer_sizes = [16, 32, 64, 128]
results = {}
histories = {}

for h2 in second_layer_sizes:
    metrics, history = train_and_eval(hidden=h2, epochs=300, lr=0.01)
    results[h2] = metrics
    histories[h2] = history

#
df_hidden2 = pd.DataFrame.from_records(
    [{"Hidden2": h2, **results[h2]} for h2 in second_layer_sizes]
).set_index("Hidden2")

def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss vs Second Layer Hidden Size"):
    keys = list(histories.keys())
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)
    for idx, k in enumerate(keys):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[k]
        ax.plot(hist["train_loss"], label="Train Loss")
        ax.plot(hist["test_loss"], label="Test Loss")
        ax.set_title(f"Hidden = {k}")
        ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
        ax.legend()

    for k in range(n, rows*cols):

```

```

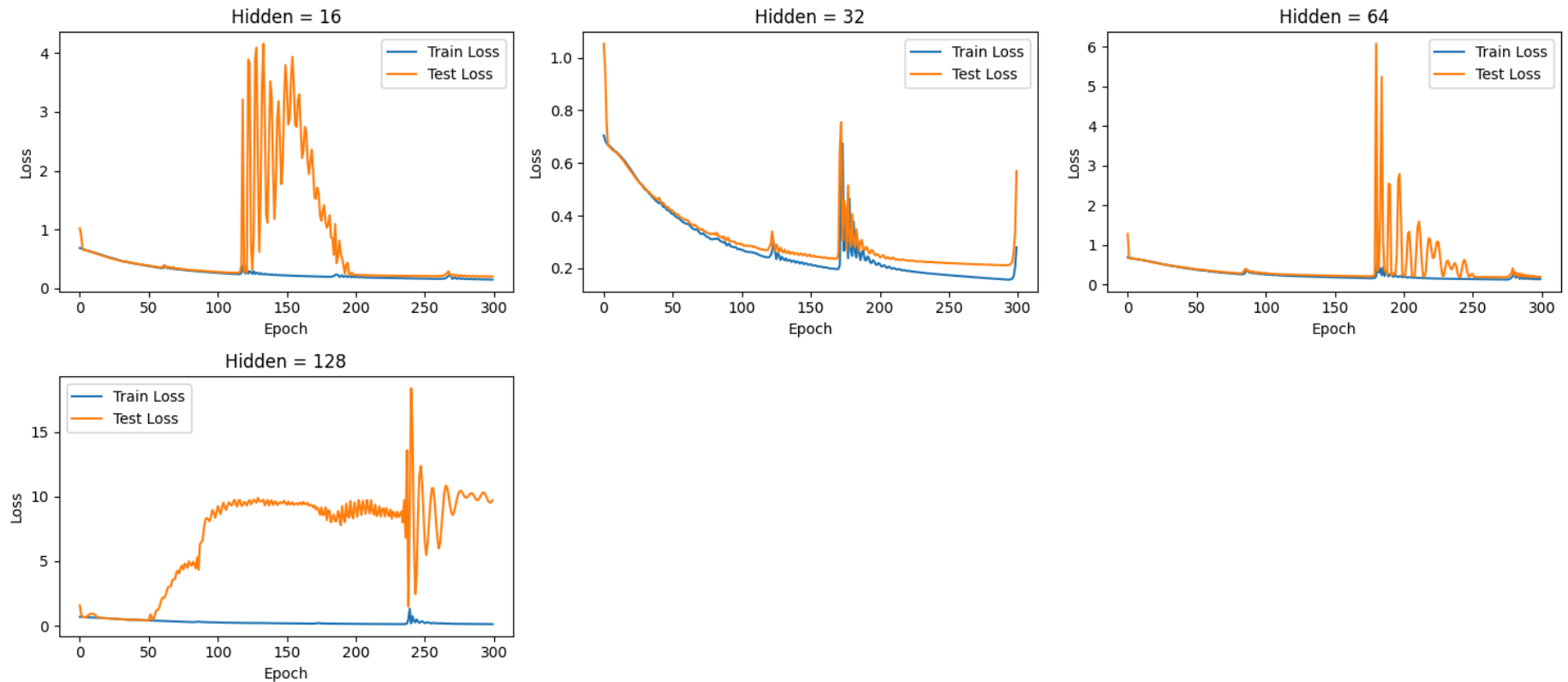
    r, c = divmod(k, cols)
    axes[r][c].axis("off")

    fig.suptitle(suptitle, y=1.02)
    plt.tight_layout()
    plt.show()

plot_loss_grid(histories)

```

Train/Test Loss vs Second Layer Hidden Size



1. Adam with a high learning rate of 0.01, when used in an MLP with two hidden layers, shows increasing fluctuations and spike instability in test loss as the size of the second hidden layer increases.

2. This is because the network has more weights and biases to adjust during each training step, so even small inaccuracies in the gradients get applied to a much larger number of parameters, amplifying their effect and causing bigger overall changes in the model's behaviour, which leads to instability in test loss.

```
In [100... def plot_metric_bars_grid(df_results, rows=3, cols=2, subtitle="Metrics vs Setting"):

    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

    for i, metric in enumerate(metrics):
        ax = ax_list[i]
        vals = df_results[metric].values
        ax.bar(idx_labels, vals, edgecolor='black')
        ax.set_title(metric)
        ax.set_xlabel("# of neurons")
        ax.set_ylabel(metric)
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)
        for x, v in enumerate(vals):
            ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

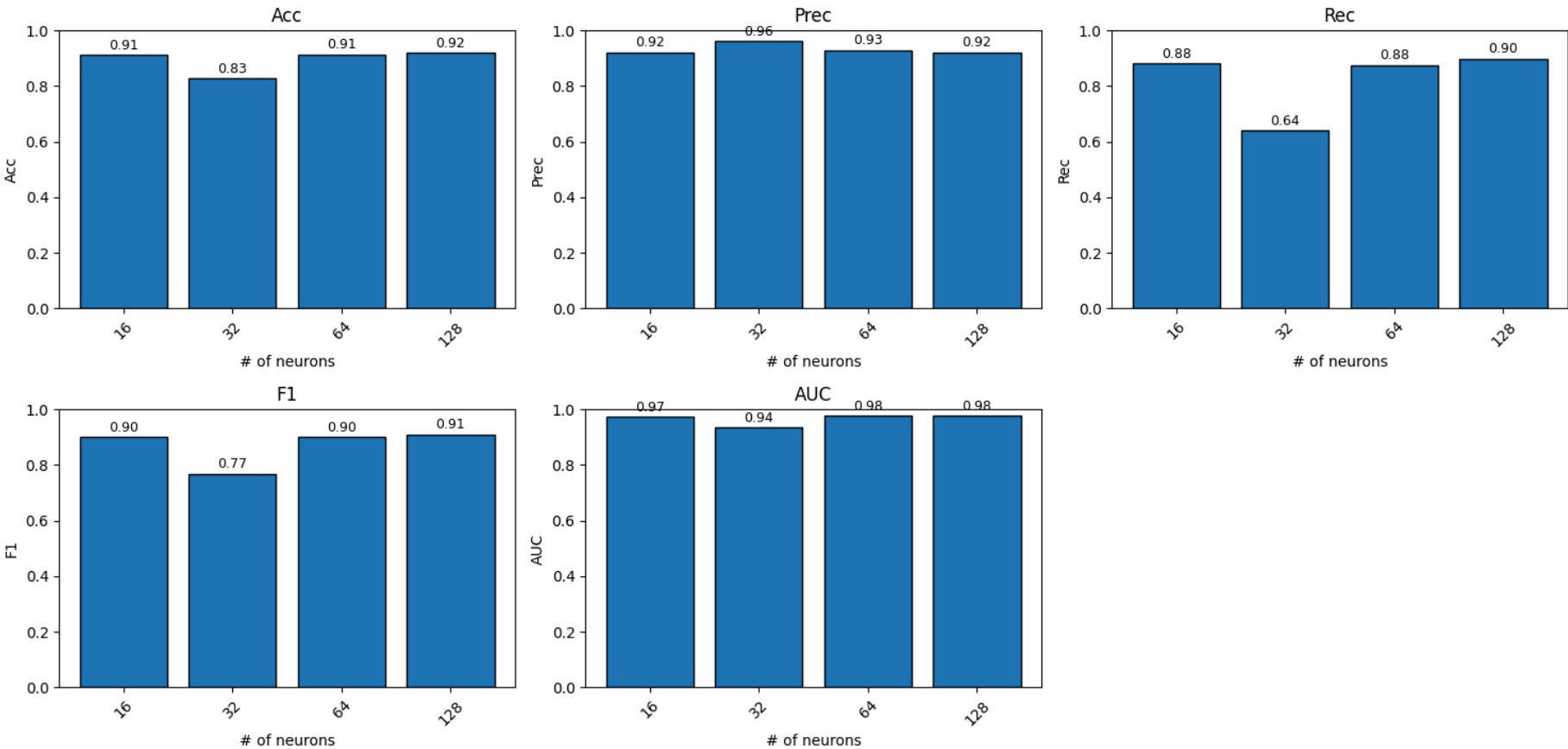
    #
    for j in range(len(metrics), rows*cols):
        ax_list[j].axis("off")

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()

#
plot_metric_bars_grid(df_hidden2, rows=2, cols=3, subtitle="Metrics vs second Hidden Layer size")

df_hidden2
```

Metrics vs second Hidden Layer size



Out[100...

	Acc	Prec	Rec	F1	AUC
Hidden2					
16	0.913218	0.921523	0.881784	0.901216	0.974755
32	0.827103	0.963046	0.639405	0.768543	0.935257
64	0.914219	0.929022	0.875836	0.901646	0.977466
128	0.920227	0.921494	0.898885	0.910049	0.977318

Even though the 128 size hidden layer had an unstable test loss, it still achieves the best classification metrics as the larger capacity allows the model to learn richer feature representations and more precise decision boundaries

Tuning for the size of second hidden layer

- sizes: 16, 32, 64, 128
- lr:0.001

```
In [101... import os
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
)
import math

torch.manual_seed(42)

class MLP2HL(nn.Module):
    def __init__(self, input_dim, hidden2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, 1) # logits
        )
    def forward(self, x):
```

```
        return self.net(x)

def train_and_eval(hidden, epochs=200, lr=best_lr):

    model = MLP2HL(X_train_tensor.shape[1], hidden2=hidden)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr,)

    train_losses, test_losses = [], []

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        logits = model(X_train_tensor)
        loss = criterion(logits, y_train_tensor)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            test_logits = model(X_test_tensor)
            test_loss = criterion(test_logits, y_test_tensor).item()
            test_losses.append(test_loss)

    model.eval()
    with torch.no_grad():
        logits = model(X_test_tensor)
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.5).float()

    y_true = y_test_tensor.cpu().numpy().ravel()
    y_pred = preds.cpu().numpy().ravel()
    y_proba = probs.cpu().numpy().ravel()

    metrics = {
        "Acc": accuracy_score(y_true, y_pred),
        "Prec": precision_score(y_true, y_pred),
        "Rec": recall_score(y_true, y_pred),
        "F1": f1_score(y_true, y_pred),
        "AUC": roc_auc_score(y_true, y_proba),
```

```

    }
    history = {"train_loss": train_losses, "test_loss": test_losses}
    return metrics, history

# second-layer-size
second_layer_sizes = [16, 32, 64, 128]
results = {}
histories = {}

for h2 in second_layer_sizes:
    metrics, history = train_and_eval(hidden=h2, epochs=300, lr=0.001)
    results[h2] = metrics
    histories[h2] = history

#
df_hidden2 = pd.DataFrame.from_records(
    [{"Hidden2": h2, **results[h2]} for h2 in second_layer_sizes]
).set_index("Hidden2")

def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss vs Second Layer Hidden Size"):
    keys = list(histories.keys())
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)
    for idx, k in enumerate(keys):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[k]
        ax.plot(hist["train_loss"], label="Train Loss")
        ax.plot(hist["test_loss"], label="Test Loss")
        ax.set_title(f"Hidden = {k}")
        ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
        ax.legend()

    for k in range(n, rows*cols):

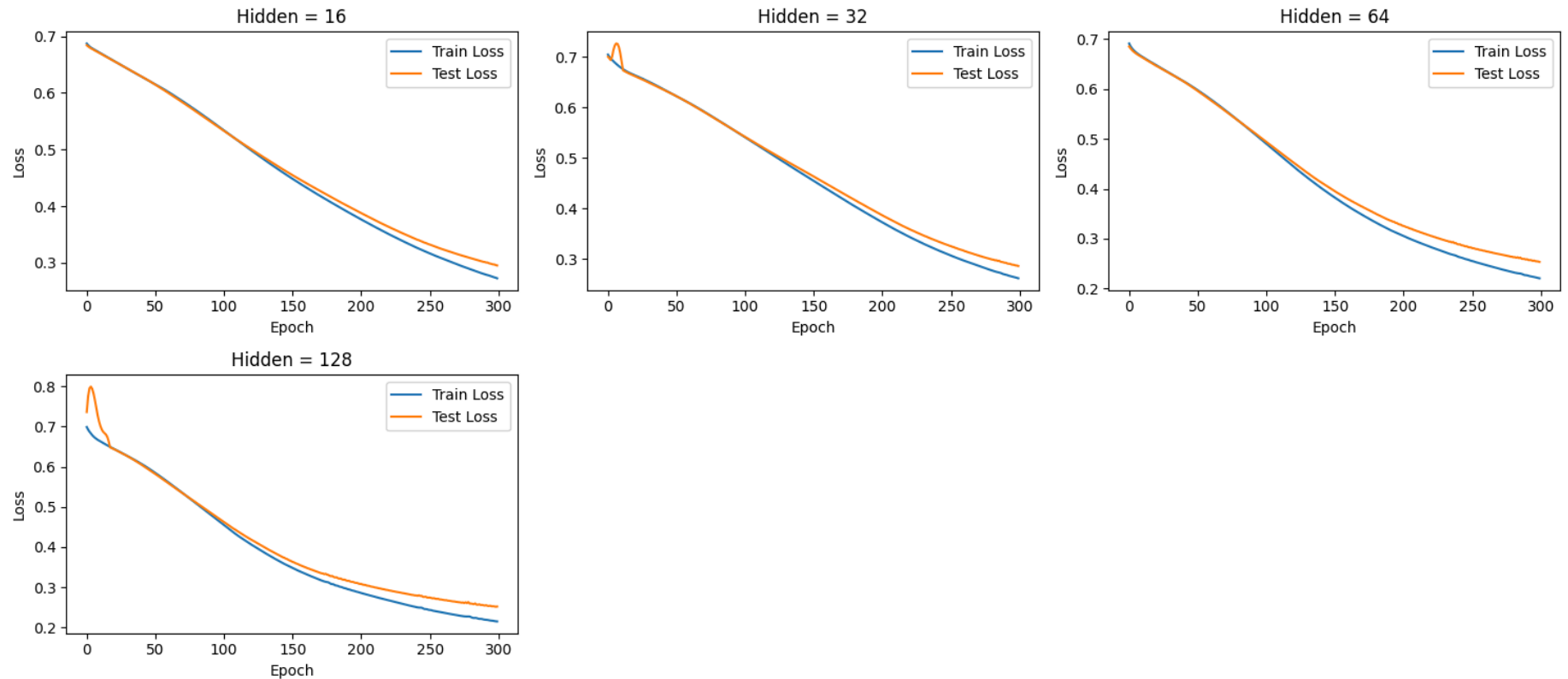
```

```
r, c = divmod(k, cols)
axes[r][c].axis("off")

fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

plot_loss_grid(histories)
```

Train/Test Loss vs Second Layer Hidden Size



When Adam with a learning rate of 0.001 is used, the train and test loss curves are now smooth and stable across all configurations because the smaller learning rate results in smaller weight updates, reducing the impact of gradient noise.

```

In [102... def plot_metric_bars_grid(df_results, rows=3, cols=2, suptitle="Metrics vs Setting"):

    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

    for i, metric in enumerate(metrics):
        ax = ax_list[i]
        vals = df_results[metric].values
        ax.bar(idx_labels, vals, edgecolor='black')
        ax.set_title(metric)
        ax.set_xlabel("# of neurons")
        ax.set_ylabel(metric)
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)
        for x, v in enumerate(vals):
            ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

    #
    for j in range(len(metrics), rows*cols):
        ax_list[j].axis("off")

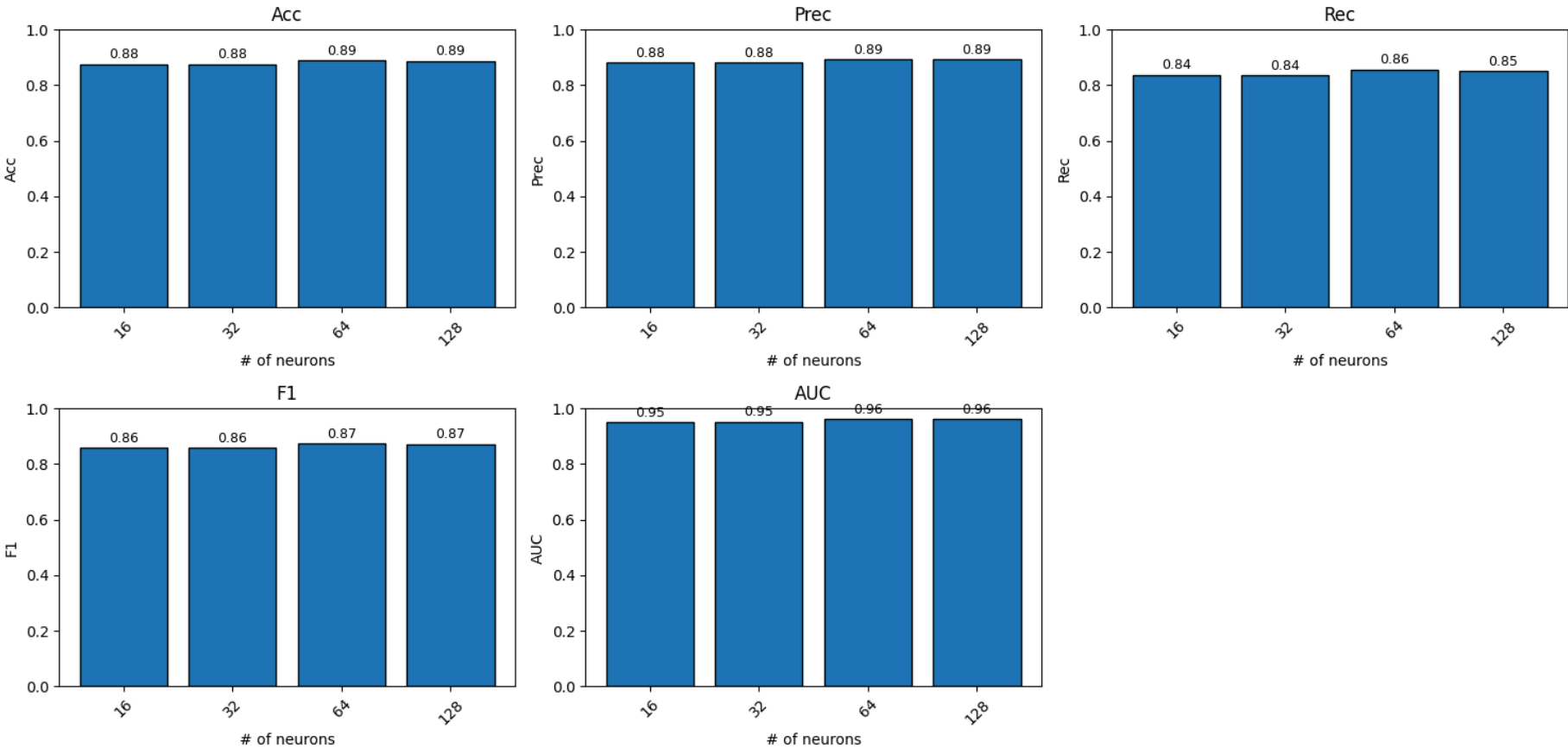
    fig.suptitle(suptitle, y=1.02)
    plt.tight_layout()
    plt.show()

#
plot_metric_bars_grid(df_hidden2, rows=2, cols=3, suptitle="Metrics vs second Hidden Layer size")

df_hidden2

```

Metrics vs second Hidden Layer size



Out[102...

	Acc	Prec	Rec	F1	AUC
Hidden2					
16	0.876502	0.881754	0.837175	0.858886	0.949977
32	0.876168	0.881661	0.836431	0.858451	0.952369
64	0.889853	0.893106	0.857249	0.874810	0.961753
128	0.887183	0.893052	0.850558	0.871287	0.961732

- Although the train and test loss curves became smooth and stable, the classification metrics did not improve significantly, suggesting that while stability in training was achieved.
- The smaller learning rate limited the model's ability to explore the parameter space enough to reach a configuration that maximizes predictive performance.

Tuning for the choice of activation function in hidden layers

- tanh vs relu function

```
In [103... import os
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
)
import math

torch.manual_seed(42)

class MLP2HL(nn.Module):
    def __init__(self, input_dim, activation):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            activation,
            nn.Linear(256, 128),
            activation,
            nn.Linear(128, 1) # logits
```

```
)  
def forward(self, x):  
    return self.net(x)  
  
def train_and_eval(act_fn, hidden=128, epochs=300, lr=0.01):  
    model = MLP2HL(X_train_tensor.shape[1], activation=act_fn)  
    criterion = nn.BCEWithLogitsLoss()  
    optimizer = optim.Adam(model.parameters(), lr=lr)  
  
    train_losses, test_losses = [], []  
  
    for _ in range(epochs):  
        model.train()  
        optimizer.zero_grad()  
        logits = model(X_train_tensor)  
        loss = criterion(logits, y_train_tensor)  
        loss.backward()  
        optimizer.step()  
        train_losses.append(loss.item())  
  
        model.eval()  
        with torch.no_grad():  
            test_logits = model(X_test_tensor)  
            test_loss = criterion(test_logits, y_test_tensor).item()  
            test_losses.append(test_loss)  
  
    #  
    model.eval()  
    with torch.no_grad():  
        logits = model(X_test_tensor)  
        probs = torch.sigmoid(logits)  
        preds = (probs >= 0.5).float()  
  
    y_true = y_test_tensor.cpu().numpy().ravel()  
    y_pred = preds.cpu().numpy().ravel()  
    y_proba = probs.cpu().numpy().ravel()  
  
    metrics = {  
        "Acc": accuracy_score(y_true, y_pred),  
        "Prec": precision_score(y_true, y_pred),  
        "Rec": recall_score(y_true, y_pred),
```

```

        "F1": f1_score(y_true, y_pred),
        "AUC": roc_auc_score(y_true, y_proba),
    }
    history = {"train_loss": train_losses, "test_loss": test_losses}
    return metrics, history

# activations
activations = {"ReLU": nn.ReLU(), "Tanh": nn.Tanh()}
results = {}
histories = {}

for name, act in activations.items():
    metrics, history = train_and_eval(act_fn=act, epochs=300, lr=0.01)
    results[name] = metrics
    histories[name] = history

# dataframe
import pandas as pd
df_act = pd.DataFrame.from_records(
    [{"Activation": k, **v} for k, v in results.items()]
).set_index("Activation")

def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss vs activation function"):
    keys = list(histories.keys()) # hidden sizes
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)
    for idx, k in enumerate(keys):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[k]
        ax.plot(hist["train_loss"], label="Train Loss")
        ax.plot(hist["test_loss"], label="Test Loss")
        ax.set_title(f"Activation function = {k}")
        ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
        ax.legend()

```

```

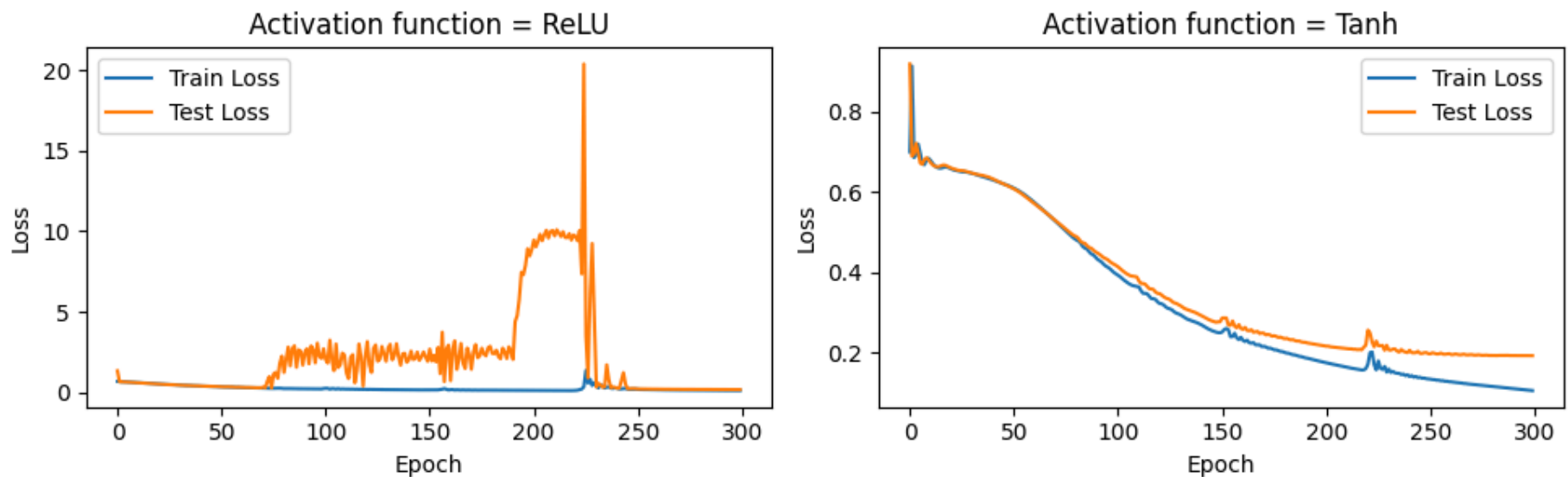
for k in range(n, rows*cols):
    r, c = divmod(k, cols)
    axes[r][c].axis("off")

fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

plot_loss_grid(histories)

```

Train/Test Loss vs activation function



1. Tanh provides smooth and stable convergence for both train and test loss, whereas ReLU shows severe instability in test loss, which eventually stabilizes after around 200 epochs.
2. The bounded activations of Tanh $[-1, 1]$ keep gradients naturally smaller and updates more controlled, making training smoother, while the large positive activations with ReLU can cause large gradients, which at a high learning rate lead to overshooting and spikes in loss.

```

In [104... def plot_metric_bars_grid(df_results, rows=3, cols=2, subtitle="Metrics vs Setting"):

    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

    for i, metric in enumerate(metrics):
        ax = ax_list[i]
        vals = df_results[metric].values
        ax.bar(idx_labels, vals, edgecolor='black')
        ax.set_title(metric)
        ax.set_xlabel("Activation Function")
        ax.set_ylabel(metric)
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)
        for x, v in enumerate(vals):
            ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

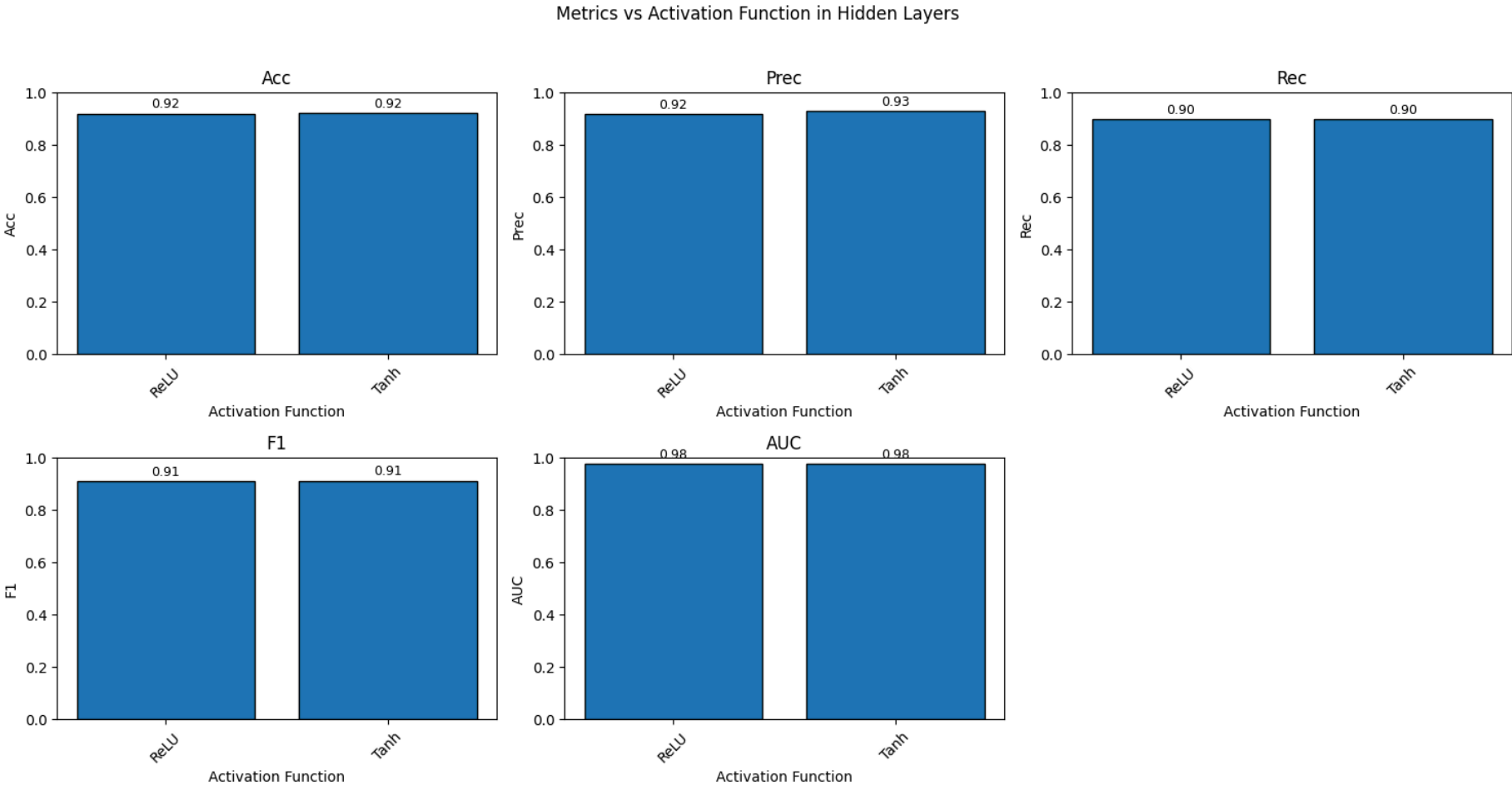
    #
    for j in range(len(metrics), rows*cols):
        ax_list[j].axis("off")

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()

plot_metric_bars_grid(df_act, rows=2, cols=3, subtitle="Metrics vs Activation Function in Hidden Layers")

df_act

```



Out[104...

	Acc	Prec	Rec	F1	AUC
Activation					
ReLU	0.919559	0.919453	0.899628	0.909433	0.978289
Tanh	0.923231	0.929176	0.897398	0.913011	0.977510

The final predictive performance of tanh is slightly better than ReLU.

Tuning the number of epochs

- 50, 100, 200, 300, 500

```
In [105... class MLP2HL(nn.Module):
    def __init__(self, input_dim, hidden2=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, 1) # logits
        )
    def forward(self, x):
        return self.net(x)

def train_and_eval_epochs(epochs, lr=0.01):

    torch.manual_seed(42)

    model = MLP2HL(X_train_tensor.shape[1], hidden2=128)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_losses, test_losses = [], []

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        logits = model(X_train_tensor)
        loss = criterion(logits, y_train_tensor)
```

```
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

    model.eval()
    with torch.no_grad():
        test_logits = model(X_test_tensor)
        test_loss = criterion(test_logits, y_test_tensor).item()
    test_losses.append(test_loss)

# final metrics on test
    model.eval()
    with torch.no_grad():
        logits = model(X_test_tensor)
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.5).float()

    y_true = y_test_tensor.cpu().numpy().ravel()
    y_pred = preds.cpu().numpy().ravel()
    y_proba = probs.cpu().numpy().ravel()

    metrics = {
        "Acc": accuracy_score(y_true, y_pred),
        "Prec": precision_score(y_true, y_pred),
        "Rec": recall_score(y_true, y_pred),
        "F1": f1_score(y_true, y_pred),
        "AUC": roc_auc_score(y_true, y_proba),
    }
    history = {"train_loss": train_losses, "test_loss": test_losses}
    return metrics, history

# epochs
epoch_counts = [50, 100, 200, 300, 500]
results, histories = {}, {}

for ep in epoch_counts:
    metrics, history = train_and_eval_epochs(epochs=ep, lr=0.01)
    results[ep] = metrics
    histories[ep] = history

#
```



```

df_epochs = pd.DataFrame.from_records(
    [{"Epochs": ep, **results[ep]} for ep in epoch_counts]
).set_index("Epochs")

# plot
import math
def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss per Epoch Count"):
    keys = list(histories.keys())
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)
    for idx, k in enumerate(keys):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[k]
        ax.plot(hist["train_loss"], label="Train Loss")
        ax.plot(hist["test_loss"], label="Test Loss")
        ax.set_title(f"Epochs = {k}")
        ax.set_xlabel("Epoch")
        ax.set_ylabel("Loss")
        ax.legend()

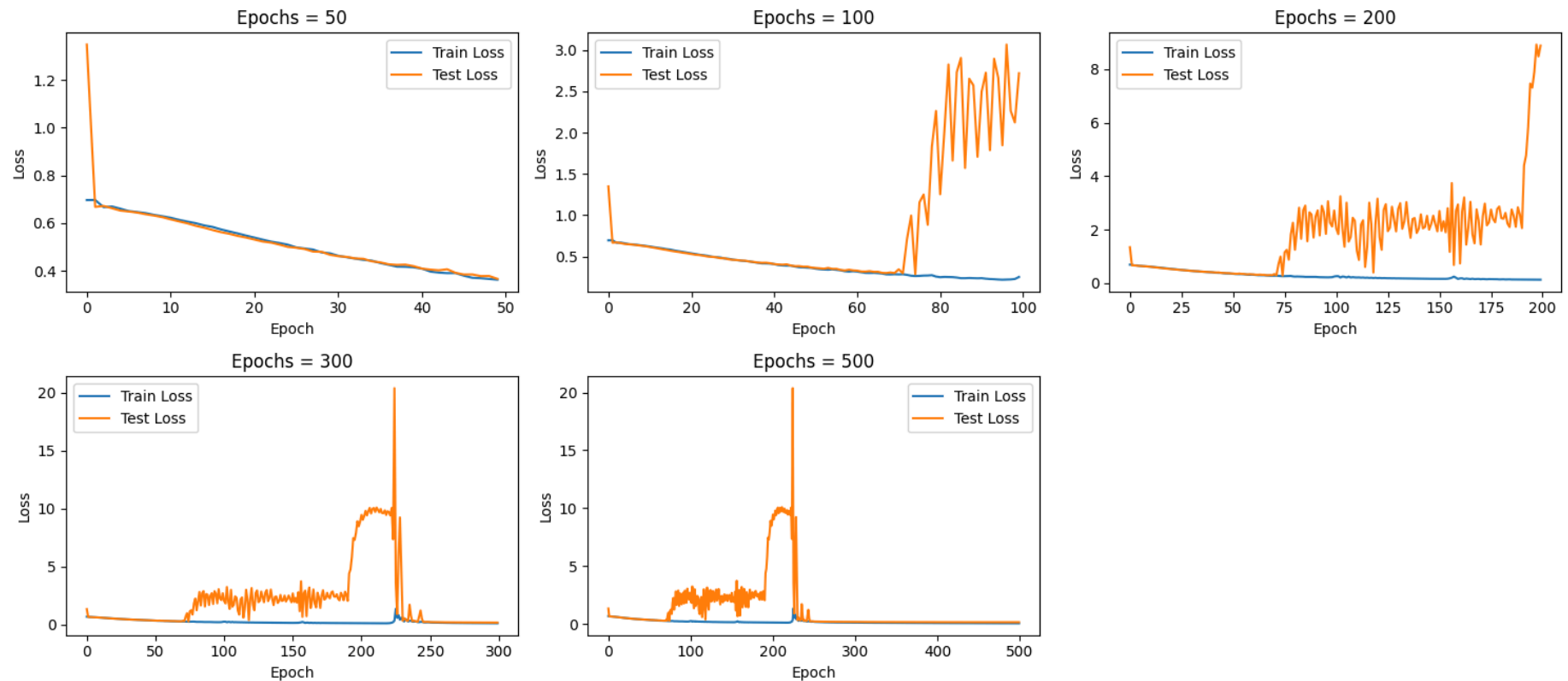
    for k in range(n, rows*cols):
        r, c = divmod(k, cols)
        axes[r][c].axis("off")

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()

plot_loss_grid(histories, max_cols=3)

```

Train/Test Loss per Epoch Count



- With a hidden layer size of 256 and 128 using Adam at 0.01, test loss spikes after ~100 epochs but regains stability by 500 epochs, with both train and test losses decreasing smoothly.

```
In [106... def plot_metric_bars_grid(df_results, rows=3, cols=2, subtitle="Metrics vs Setting"):

    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)

    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

    for i, metric in enumerate(metrics):
```

```
ax = ax_list[i]
vals = df_results[metric].values
ax.bar(idx_labels, vals, edgecolor='black')
ax.set_title(metric)
ax.set_xlabel("No of epochs")
ax.set_ylabel(metric)
ax.set_ylim(0, 1)
ax.tick_params(axis='x', rotation=45)
for x, v in enumerate(vals):
    ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

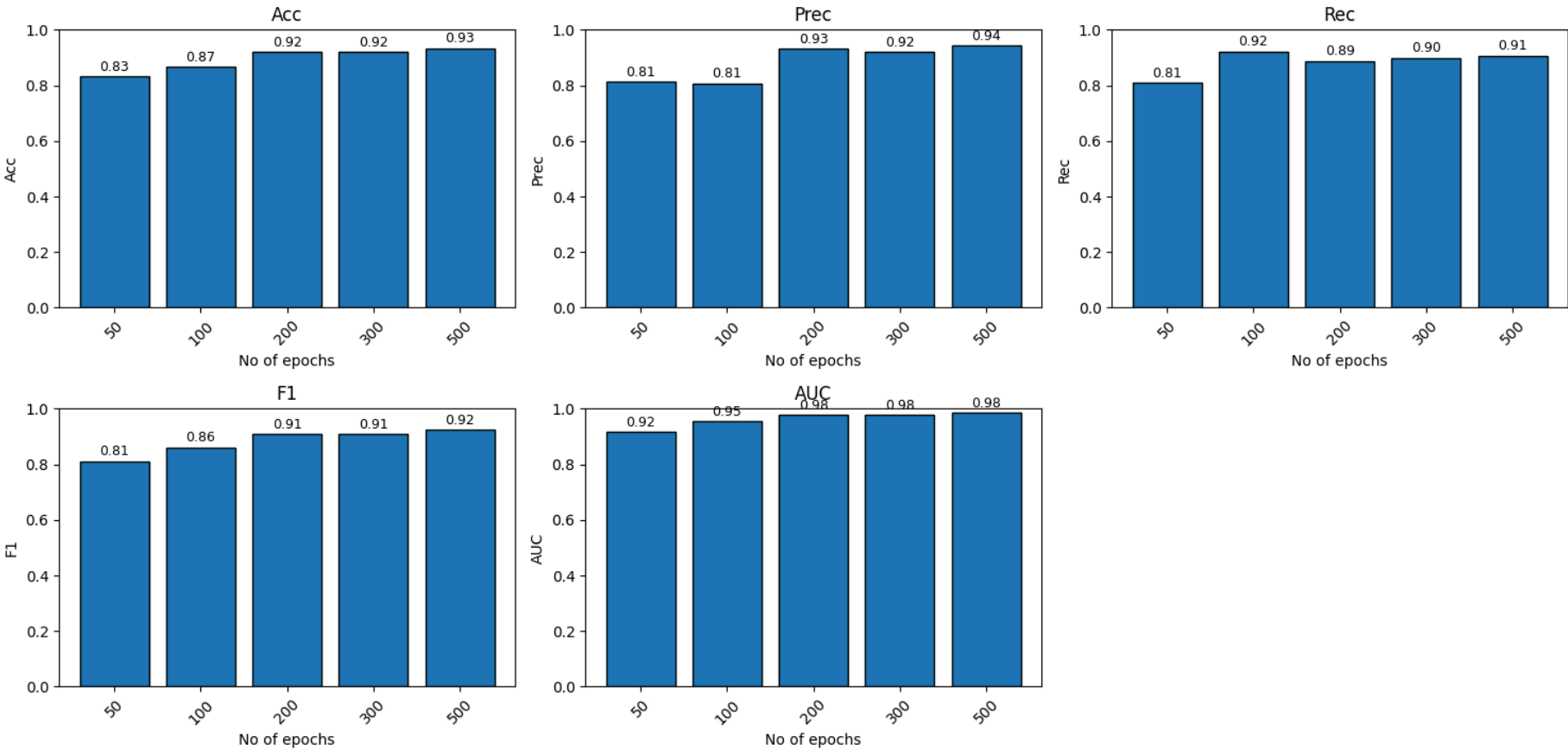
# hide unused axes if metrics < rows*cols
for j in range(len(metrics), rows*cols):
    ax_list[j].axis("off")

fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

# Call
plot_metricBars_grid(df_epochs, rows=2, cols=3, suptitle="Metrics vs No of epochs")

df_epochs
```

Metrics vs No of epochs



Out[106]...

	Acc	Prec	Rec	F1	AUC
Epochs					
50	0.831108	0.813294	0.809665	0.811475	0.916664
100	0.865821	0.806766	0.921933	0.860514	0.954790
200	0.919559	0.930577	0.886989	0.908260	0.976772
300	0.919559	0.919453	0.899628	0.909433	0.978289
500	0.933578	0.943498	0.906320	0.924535	0.984269

The best predictive performance is for 500 epochs

Tuning the batch size

- Batch size :16,32,64

Earlier, we were optimizing using all training samples in each epoch (full-batch gradient descent). Now, we switch to mini-batch training with batch sizes of 16, 32, and 64, meaning that within each epoch, the model parameters are updated multiple times i.e. once after processing each batch-size worth of elements which allows faster and more frequent weight updates.

In [107]...

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
import os
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(42)
```

```

class MLP2HL(nn.Module):
    def __init__(self, input_dim, hidden2=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, 1) # logits
        )
    def forward(self, x):
        return self.net(x)

best_lr = 0.01

def train_and_eval_batchsize(batch_size, epochs=200, weight_decay=0.0, lr=best_lr):

    model = MLP2HL(X_train_tensor.shape[1], hidden2=128)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)

    # DataLoader for mini-batch training
    train_ds = TensorDataset(X_train_tensor, y_train_tensor)
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, drop_last=False)

    train_losses, test_losses = [], []

    for _ in range(epochs):
        model.train()
        total_loss, total_n = 0.0, 0
        for xb, yb in train_loader:
            optimizer.zero_grad()
            logits = model(xb)
            loss = criterion(logits, yb)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * xb.size(0)
            total_n += xb.size(0)
        train_losses.append(total_loss / total_n)

```

```

    # test loss (full-batch)
    model.eval()
    with torch.no_grad():
        test_logits = model(X_test_tensor)
        test_loss = criterion(test_logits, y_test_tensor).item()
    test_losses.append(test_loss)

#
model.eval()
with torch.no_grad():
    logits = model(X_test_tensor)
    probs = torch.sigmoid(logits)
    preds = (probs >= 0.5).float()

y_true = y_test_tensor.cpu().numpy().ravel()
y_pred = preds.cpu().numpy().ravel()
y_proba = probs.cpu().numpy().ravel()

metrics = {
    "Acc": accuracy_score(y_true, y_pred),
    "Prec": precision_score(y_true, y_pred),
    "Rec": recall_score(y_true, y_pred),
    "F1": f1_score(y_true, y_pred),
    "AUC": roc_auc_score(y_true, y_proba),
}
history = {"train_loss": train_losses, "test_loss": test_losses}
return metrics, history

# batch sizes
batch_sizes = [16, 32, 64]
results, histories = {}, {}

for bs in batch_sizes:
    metrics, history = train_and_eval_batchsize(batch_size=bs, epochs=20, weight_decay=0.0)
    results[bs] = metrics
    histories[bs] = history

#
df_bs = pd.DataFrame.from_records(
    [{"BatchSize": bs, **results[bs]} for bs in batch_sizes]
).set_index("BatchSize")

```

```
import math

def plot_loss_grid(histories, max_cols=3, subtitle="Train/Test Loss per Batch Size"):
    keys = list(histories.keys()) # hidden sizes
    n = len(keys)
    cols = min(max_cols, n)
    rows = math.ceil(n / cols)

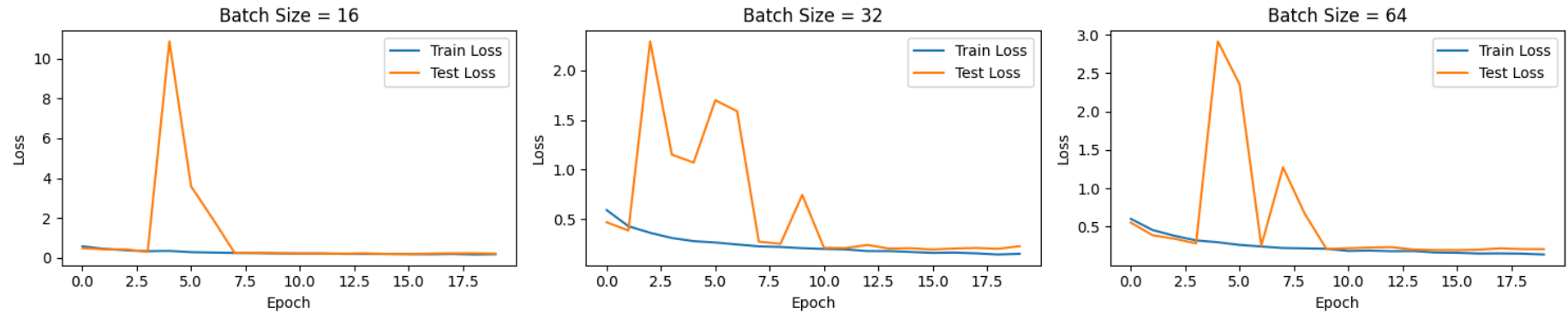
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows), squeeze=False)
    for idx, k in enumerate(keys):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        hist = histories[k]
        ax.plot(hist["train_loss"], label="Train Loss")
        ax.plot(hist["test_loss"], label="Test Loss")
        ax.set_title(f"Batch Size = {k}")
        ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
        ax.legend()

    for k in range(n, rows*cols):
        r, c = divmod(k, cols)
        axes[r][c].axis("off")

    fig.suptitle(subtitle, y=1.02)
    plt.tight_layout()
    plt.show()

plot_loss_grid(histories)
```


Train/Test Loss per Batch Size



Since there are multiple updates in each epoch, we train only for 20 epochs, and the updates become noisier because each mini-batch provides only a partial view of the training data.

```
In [108... def plot_metric_bars_grid(df_results, rows=3, cols=2, subtitle="Metrics vs Setting"):

    metrics = list(df_results.columns)
    idx_labels = df_results.index.astype(str)

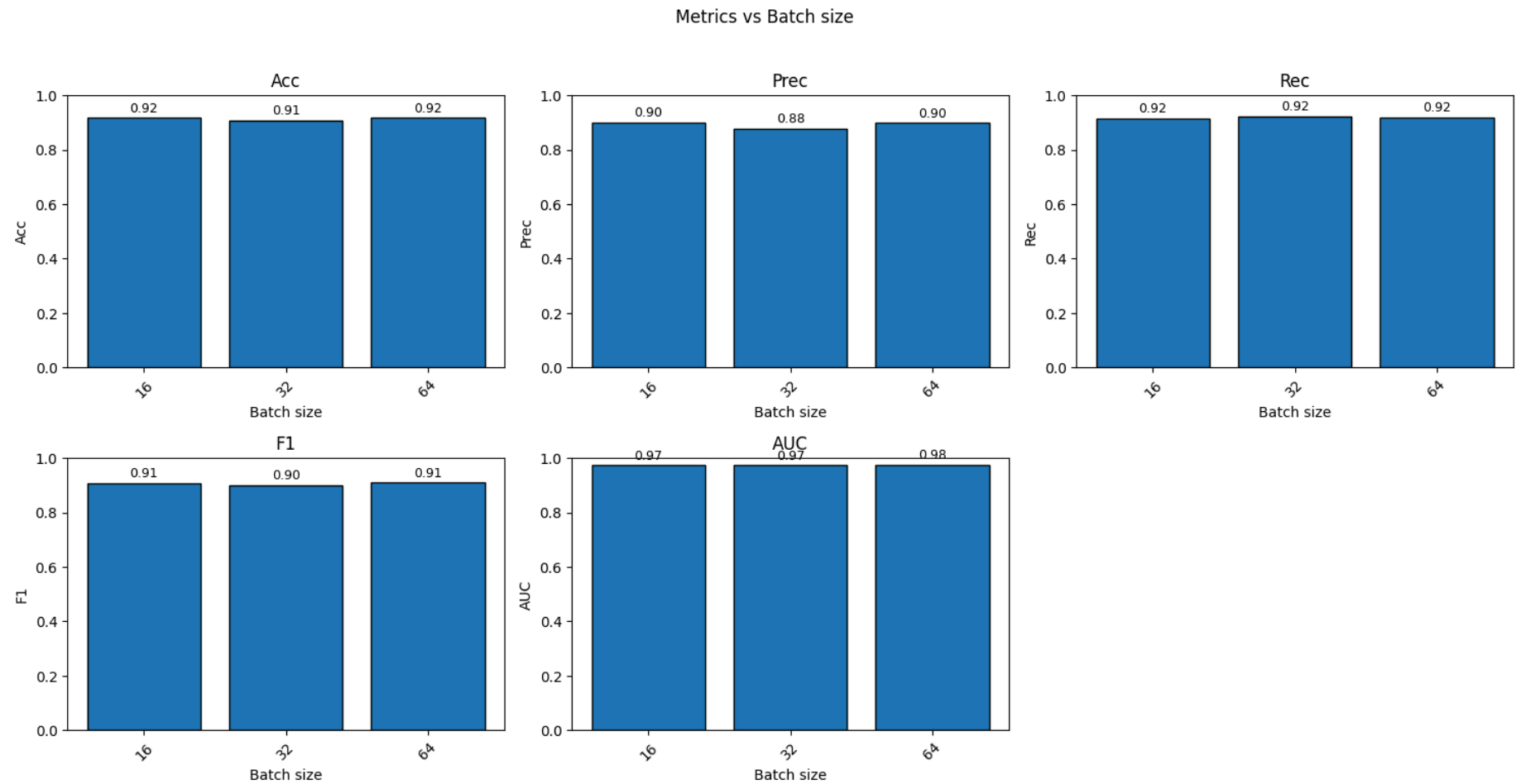
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.8*rows), squeeze=False)
    ax_list = axes.ravel()

    for i, metric in enumerate(metrics):
        ax = ax_list[i]
        vals = df_results[metric].values
        ax.bar(idx_labels, vals, edgecolor='black')
        ax.set_title(metric)
        ax.set_xlabel("Batch size")
        ax.set_ylabel(metric)
        ax.set_ylim(0, 1)
        ax.tick_params(axis='x', rotation=45)
        for x, v in enumerate(vals):
            ax.text(x, v + 0.01, f"{v:.2f}", ha='center', va='bottom', fontsize=9)

    for j in range(len(metrics), rows*cols):
        ax_list[j].axis("off")
```

```
fig.suptitle(suptitle, y=1.02)
plt.tight_layout()
plt.show()

#
plot_metric_bars_grid(df_bs, rows=2, cols=3, suptitle="Metrics vs Batch size")
df_bs
```



Out[108...

	Acc	Prec	Rec	F1	AUC
BatchSize					
16	0.917223	0.901832	0.915242	0.908487	0.973843
32	0.908545	0.878445	0.924164	0.900725	0.973715
64	0.917891	0.899056	0.920446	0.909625	0.975524

The performance is comparable for batch size 16 and 64

Training the final MLP

- Adam optimizer with lr 0.01
- Batch size 16
- 2 Hidden layer 256,128
- ReLU activation functions
- epochs 50

In [109...

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

# DataLoader
batch_size = 16
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

#
```

```

class MLP(nn.Module):
    def __init__(self, input_dim):
        super(MLP, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(), #nn.ReLU(), # nn.Tanh()
            nn.Linear(256, 128),
            nn.ReLU(), #nn.ReLU(),
            nn.Linear(128, 1) # logits
        )
    def forward(self, x):
        return self.net(x)

#
model = MLP(X_train_tensor.shape[1])
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Training loop
epochs = 50
for _ in range(epochs):
    model.train()
    for xb, yb in train_loader:
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()

# Evaluation
model.eval()
with torch.no_grad():
    logits = model(X_test_tensor)
    probs = torch.sigmoid(logits)
    preds = (probs >= 0.5).float()

y_true = y_test_tensor.numpy()
y_pred = preds.numpy()
y_proba = probs.numpy()

metrics = {

```

```

    "Acc": accuracy_score(y_true, y_pred),
    "Prec": precision_score(y_true, y_pred),
    "Rec": recall_score(y_true, y_pred),
    "F1": f1_score(y_true, y_pred),
    "AUC": roc_auc_score(y_true, y_proba),
}

# Output
df_results = pd.DataFrame([metrics], index=["MLP"])
df_results

```

Out[109]...

	Acc	Prec	Rec	F1	AUC
MLP	0.935915	0.936412	0.919703	0.927982	0.980477

Comparing with baselines Non-linear models

```

In [110... from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

# Baseline models
models = {
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100, max_depth=10),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=7)
}

# Collect metrics for baselines
baseline_results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_proba = model.predict_proba(X_test_scaled)[:, 1]

    baseline_results[name] = {

```

```
        "Acc": accuracy_score(y_test, y_pred),
        "Prec": precision_score(y_test, y_pred),
        "Rec": recall_score(y_test, y_pred),
        "F1": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_proba)
    }

#
df_baselines = pd.DataFrame.from_dict(baseline_results, orient="index")

# Append
df_final = pd.concat([df_results, df_baselines], axis=0)

df_final
```

Out[110...

	Acc	Prec	Rec	F1	AUC
MLP	0.935915	0.936412	0.919703	0.927982	0.980477
Random Forest	0.855140	0.896432	0.765799	0.825982	0.943216
Decision Tree	0.827437	0.805310	0.811896	0.808589	0.825996
KNN	0.907877	0.908952	0.883271	0.895928	0.964621

- The MLP outperforms all other models across every metric, achieving the highest accuracy, precision, recall, F1-score, and AUC.
- This indicates its superior ability to capture complex patterns in the data compared to Random Forest, Decision Tree, and KNN.

In []:

In []: